the developers to implement, users explain what they need to get out of the system and how it might help them achieve their objectives. And rather than accepting a list of feature requests from the users with no questions asked, teams try to understand the core business goals underlying the project, proposing only features that can be demonstrated to support these business goals. This constant focus on delivering business value means that teams can deliver more useful features earlier and with less wasted effort.

WORK TOGETHER TO SPECIFY FEATURES

> A complex problem, like discovering ways to delight clients, is best solved by a cognitively diverse group of people that is given responsibility for solving the problem, self-organizes, and works together to solve it.
>
> Stephen Denning, *The Leader's Guide to Radical Management* (Jossey-Bass, 2010)

BDD is a highly collaborative practice, both between users and the development team, and within the team itself. Business analysts, developers, and testers work together with the end users to define and specify features, and team members draw ideas from their individual experience and know-how. This approach is highly efficient.

In a more traditional approach, when business analysts simply relay their understanding of the users' requirements to the rest of the team, there is a high risk of misinterpretation and lost information.

If you ask users to write up what they want, they'll typically give you a set of detailed requirements that matches how they envisage the solution. In other words, users will not tell you *what they need*; rather, *they'll design a solution for you*. I've seen many business analysts fall into the same trap, simply because they've been trained to write specifications that way. The problem with this approach is twofold: not only will they fail to benefit from the development team's expertise in software design, but they're effectively binding the development team to a particular solution, which may not be the optimal one in business or technical terms. In addition, developers can't use their technical know-how to help deliver a technically superior design, and testers don't get the opportunity to comment on the testability of the specifications until the end of the project.

For example, the "Transfer funds to an overseas account" feature involves many user-experience and technical considerations. How can you display the constantly changing exchange rates to the client? When and how are the fees calculated and shown to the client? For how long can you guarantee a proposed exchange rate? How can you verify that the right exchange rate is being used? All of these considerations will have an impact on the design, implementation, and cost of the feature and can change the way the business analysts and business stakeholders originally imagined the solution.

When teams practice BDD, on the other hand, team members build up a shared appreciation of the users' needs, as well as a sense of common ownership and engagement in the solution.